INDICADOR DE LOGRO IDL3

Big Data Aplicada

Elaborado por:

APAZA PEREZ OSCAR GONZALO
CABEZAS HUANIO RUBEN KELVIN
RUIZ ALVA JERSON ENMANUEL
PONCE DE LEON TORRES FABYOLA KORAYMA

Solicitado por:

ORIZANO SALVADOR SERGIO VICTOR

Huancayo, 2025

Tabla de contenidos

1 Verificando Hadoop

```
!hadoop version
# start-all.sh desde usuario hadoop
# jps
```

```
Hadoop 3.4.1
Source code repository https://github.com/apache/hadoop.git -r 4d7825309348956336b8f06a08322b78422
Compiled by mthakur on 2024-10-09T14:57Z
Compiled on platform linux-x86_64
Compiled with protoc 3.23.4
From source with checksum 7292fe9dba5e2e44e3a9f763fce3e680
This command was run using /home/hadoop/hadoop/share/hadoop/common/hadoop-common-3.4.1.jar
```

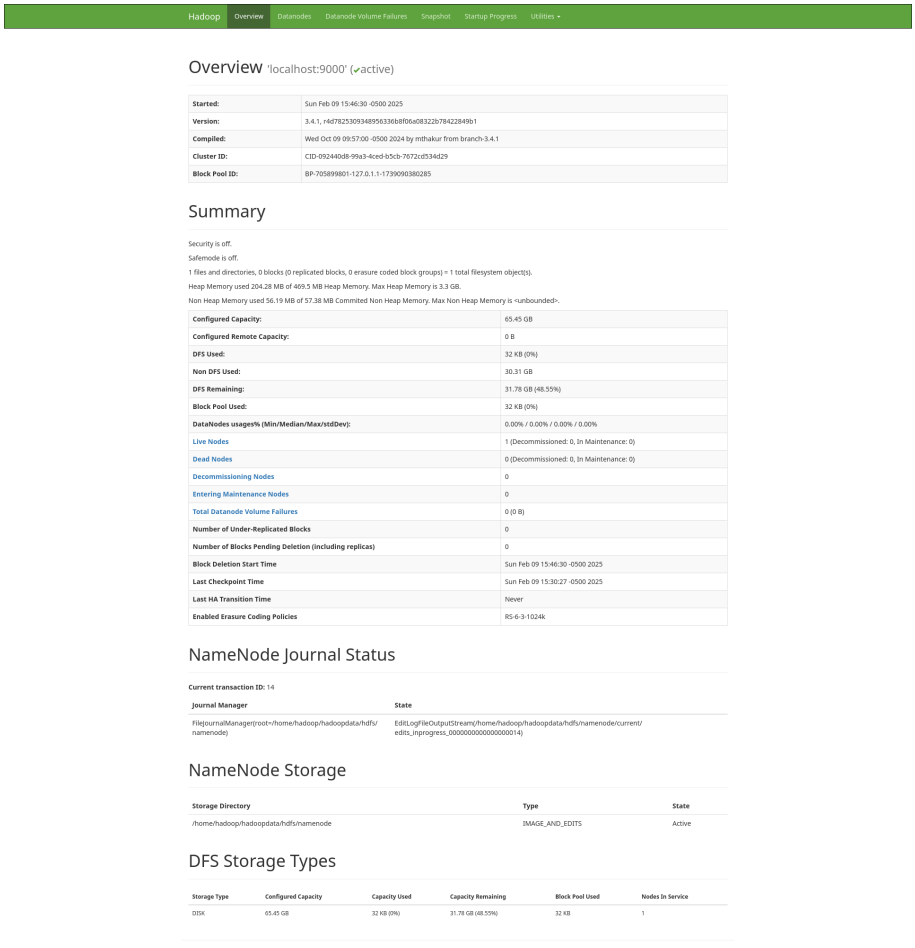

Figura 1: Namenode http://localhost:9870/

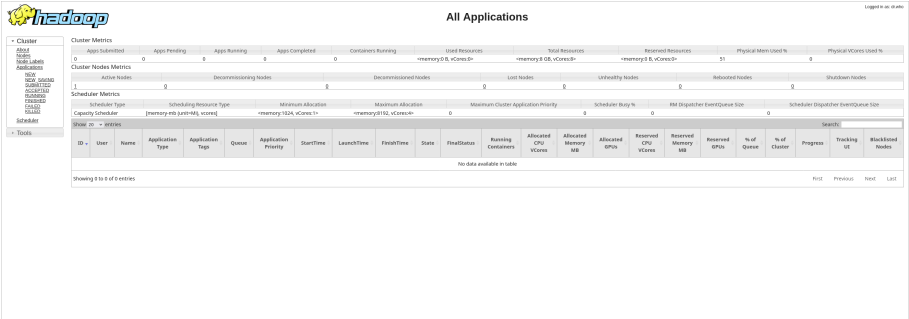

Figura 2: Cluster http://localhost:8088/

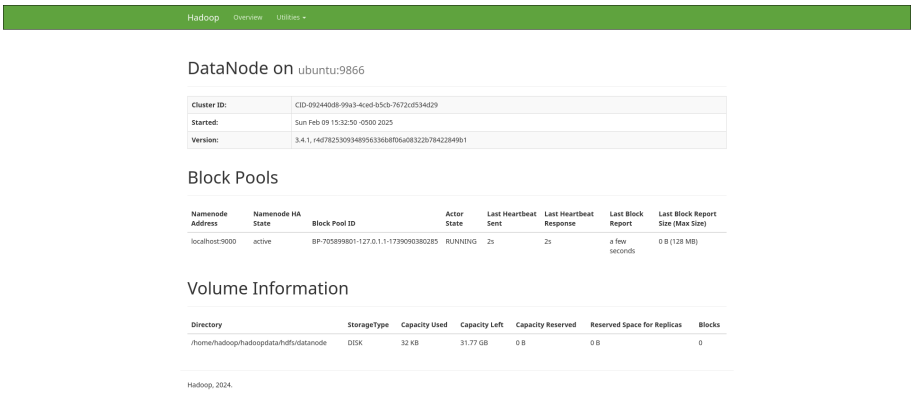

Figura 3: Datanode http://localhost:9864

2 Verificando Spark

```
!spark-shell --version
# start-master.sh desde usuario hadoop
# jps | grep Master
```

```
25/02/09 18:25:14 WARN Utils: Your hostname, ubuntu resolves to a loopback address: 127.0.1.1; usi
25/02/09 18:25:14 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.5.4
      /_/

Using Scala version 2.12.18, OpenJDK 64-Bit Server VM, 1.8.0_442
Branch HEAD
Compiled by user yangjie01 on 2024-12-17T04:51:46Z
Revision a6f220d951742f4074b37772485ee0ec7a774e7d
Url https://github.com/apache/spark
Type --help for more information.
```

Figura 4: Spark Master at spark://ubuntu:7077

3 Subiendo sales_data.csv a hdfs

```
# En el usuario hadoop
# sudo cp /home/jerson/Documentos/quarto/sales_data.csv /home/hadoop/hadoopdata/
# hdfs dfs -mkdir /sales
# hdfs dfs -put /home/hadoop/hadoopdata/sales_data.csv /sales
!hadoop fs -ls /sales
```

```
Found 1 items
-rw-r--r--   1 hadoop supergroup      31219 2025-02-09 17:20 /sales/sales_data.csv
```

4 Iniciando la sesion de Spark

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import sum as spark_sum

def create_spark_session():
    """
    Crea una sesión de Spark configurada para trabajar con HDFS
    """
    spark = SparkSession.builder \
        .appName("HDFSProcessing") \
        .config("spark.driver.host", "192.168.18.80") \
        .config("spark.driver.bindAddress", "192.168.18.80") \
        .config("spark.hadoop.fs.defaultFS", "hdfs://localhost:9000") \
        .config("spark.executor.memory", "2g") \
        .config("spark.driver.memory", "2g") \
        .getOrCreate()

    spark.sparkContext.setLogLevel("ERROR")
    return spark

def read_and_process_sales():
    try:
        # Crear sesión Spark
        spark = create_spark_session()

        # Ruta del archivo en HDFS
        hdfs_path = "hdfs:///sales/sales_data.csv"
```

```
        # Leer el archivo CSV desde HDFS
        df = spark.read.option("header", "true") \
                       .option("inferSchema", "true") \
                       .csv(hdfs_path)

        # Realizar el análisis
        sales_by_region = df.groupBy("Region") \
                            .agg(spark_sum("Sales").alias("TotalSales")) \
                            .orderBy("Region")

        # Mostrar resultados
        print("Ventas por región:")
        sales_by_region.show()

        return sales_by_region

    except Exception as e:
        print(f"Error en el procesamiento: {str(e)}")
        raise
    finally:
        spark.stop()

if __name__ == "__main__":
    read_and_process_sales()
```

```
25/02/09 18:25:16 WARN Utils: Your hostname, ubuntu resolves to a loopback address: 127.0.1.1; usi
25/02/09 18:25:16 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/02/09 18:25:16 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform...

Ventas por región:
+------+----------+
|Region|TotalSales|
+------+----------+
|  East|    296780|
| North|    294153|
| South|    351882|
|  West|    313536|
+------+----------+
```

5 Configurar un esquema NoSQL

```
from pyspark.sql import SparkSession
from pymongo import MongoClient
from datetime import datetime
import pandas as pd

def create_spark_session():
    """Crear sesión Spark para leer datos"""
    return SparkSession.builder \
        .appName("SalesETL") \
        .config("spark.driver.host", "192.168.18.80") \
        .config("spark.driver.bindAddress", "192.168.18.80") \
        .config("spark.hadoop.fs.defaultFS", "hdfs://localhost:9000") \
        .getOrCreate()

def date_to_datetime(date_obj):
```

```
    """Convierte un objeto date a datetime"""
    if isinstance(date_obj, datetime):
        return date_obj
    return datetime.combine(date_obj, datetime.min.time())

def process_and_load_to_mongo():
    # Configuración MongoDB
    client = MongoClient('mongodb://localhost:27017/')
    db = client['sales_db']

    # Limpiar colecciones existentes
    db.sales.drop()
    db.region_stats.drop()
    db.product_stats.drop()

    # Crear colecciones con índices optimizados
    sales_collection = db['sales']
    region_stats = db['region_stats']
    product_stats = db['product_stats']

    # Crear índices para búsquedas rápidas
    sales_collection.create_index([("region", 1)])
    sales_collection.create_index([("product", 1)])
    sales_collection.create_index([("date", 1)])

    try:
        # Leer datos con Spark
        spark = create_spark_session()
        df = spark.read.csv("hdfs:///sales/sales_data.csv", header=True, inferSchema=True)

        # Convertir a pandas para procesamiento
        pandas_df = df.toPandas()

        # Procesar y cargar ventas individuales
        for _, row in pandas_df.iterrows():
            # Convertir la fecha a datetime
            date_obj = date_to_datetime(row['Date'])

            sale_doc = {
                "region": row['Region'],
                "product": row['Product'],
                "date": date_obj,
                "sales": float(row['Sales']),
                "year_month": date_obj.strftime('%Y-%m')
            }
            sales_collection.insert_one(sale_doc)

        # Agregaciones por región
        pipeline_region = [
            {"$group": {
                "_id": "$region",
                "total_sales": {"$sum": "$sales"},
                "avg_sales": {"$avg": "$sales"},
                "sales_count": {"$sum": 1},
                "products": {"$addToSet": "$product"}
            }}
        ]
        region_results = list(sales_collection.aggregate(pipeline_region))
```

```
        region_stats.insert_many(region_results)

        # Agregaciones por producto
        pipeline_product = [
            {"$group": {
                "_id": "$product",
                "total_sales": {"$sum": "$sales"},
                "regions_sold": {"$addToSet": "$region"},
                "first_sale": {"$min": "$date"},
                "last_sale": {"$max": "$date"}
            }}
        ]
        product_results = list(sales_collection.aggregate(pipeline_product))
        product_stats.insert_many(product_results)

        return client

    except Exception as e:
        print(f"Error específico durante el procesamiento: {type(e).__name__} - {str(e)}")
        raise

def print_collection_contents(client):
    """Función para verificar los datos cargados"""
    db = client['sales_db']

    print("\nContenido de la colección 'sales':")
    for doc in db.sales.find().limit(3):
        print(doc)

    print("\nEstadísticas por región:")
    for doc in db.region_stats.find():
        print(doc)

    print("\nEstadísticas por producto:")
    for doc in db.product_stats.find():
        print(doc)

def verify_data_types(client):
    """Verificar los tipos de datos en la colección"""
    db = client['sales_db']
    first_doc = db.sales.find_one()
    if first_doc:
        print("\nTipos de datos en el primer documento:")
        for key, value in first_doc.items():
            print(f"{key}: {type(value)}")

if __name__ == "__main__":
    try:
        client = process_and_load_to_mongo()
        print("Datos cargados exitosamente")
        print_collection_contents(client)
        verify_data_types(client)
    except Exception as e:
        print(f"Error durante el procesamiento: {str(e)}")
    finally:
        if 'client' in locals():
            client.close()
```

```
Datos cargados exitosamente
```

```
Contenido de la colección 'sales':
{'_id': ObjectId('67a93961aca4780347dce351'), 'region': 'North', 'product': 'Product_D', 'date': d
{'_id': ObjectId('67a93961aca4780347dce352'), 'region': 'West', 'product': 'Product_C', 'date': da
{'_id': ObjectId('67a93961aca4780347dce353'), 'region': 'East', 'product': 'Product_C', 'date': da

Estadísticas por región:
{'_id': 'North', 'total_sales': 294153.0, 'avg_sales': 1262.4592274678112, 'sales_count': 233, 'pr
{'_id': 'West', 'total_sales': 313536.0, 'avg_sales': 1259.1807228915663, 'sales_count': 249, 'pro
{'_id': 'East', 'total_sales': 296780.0, 'avg_sales': 1262.8936170212767, 'sales_count': 235, 'pro
{'_id': 'South', 'total_sales': 351882.0, 'avg_sales': 1243.399293286219, 'sales_count': 283, 'pro

Estadísticas por producto:
{'_id': 'Product_C', 'total_sales': 358835.0, 'regions_sold': ['East', 'West', 'South', 'North'],
{'_id': 'Product_A', 'total_sales': 275695.0, 'regions_sold': ['South', 'West', 'East', 'North'],
{'_id': 'Product_B', 'total_sales': 286188.0, 'regions_sold': ['South', 'West', 'East', 'North'],
{'_id': 'Product_D', 'total_sales': 335633.0, 'regions_sold': ['East', 'West', 'South', 'North'],

Tipos de datos en el primer documento:
_id: <class 'bson.objectid.ObjectId'>
region: <class 'str'>
product: <class 'str'>
date: <class 'datetime.datetime'>
sales: <class 'float'>
year_month: <class 'str'>
```